

Лабораторная работа №5

основные вероятностные алгоритм проверки чисел

11 Марта 2026 - Ли Хан

2026

Российский университет дружбы народов, Москва, Россия

Цель работы

В рамках данной лабораторной работы я реализовал на языке Python три основных вероятностных алгоритма проверки чисел на простоту. Основная задача заключалась в программном воплощении тестов Ферма, Соловья-Штрассена и Миллера-Рабина. Для работы теста Соловья-Штрассена был отдельно реализован алгоритм вычисления символа Якоби.

Детальный разбор алгоритмов

- Проверка ввода: В начале кода я отсеиваю четные числа и числа меньше 5, чтобы обеспечить корректную работу алгоритма.
- Модульное возведение в степень: Я использую функцию `pow(a, n - 1, n)`. Это позволяет эффективно вычислять $a^{n-1} \pmod n$ с использованием алгоритма быстрого возведения в степень.
- Логика проверки: Если хотя бы в одной из t итераций результат не равен 1, код определяет число как составное. Это основано на малой теореме Ферма.

- Итерация: Я использую цикл `while a != 0` для последовательного упрощения вычислений.
- Обработка множителей: С помощью цикла `while a1 % 2 == 0` извлекаются множители двойки. Затем знак g обновляется в зависимости от $n \pmod{8}$.
- Закон взаимности: Условие `if` реализует квадратичный закон взаимности: если n и a_1 дают остаток 3 при делении на 4, знак меняется на противоположный.

- Проверка по критерию Эйлера: Ключевым шагом является вычисление $\text{pow}(a, (n - 1) // 2, n)$.
- Сравнение с символом Якоби: Результат возведения в степень сравнивается со значением, возвращаемым функцией `jacobi_symbol(a, n)`.
- Критерий простоты: Число проходит тест только в том случае, если результат степени и символ Якоби сравнимы по модулю n . Это делает тест более надежным, чем тест Ферма.

- Разложение числа: Сначала код раскладывает $n - 1$ на множители вида $2^s \cdot r$.
- Начальная проверка: Вычисляется $y = \text{pow}(a, r, n)$. Если y равно 1 или $n - 1$, тест считается пройденным для данной итерации.
- Поиск нетривиальных корней: Если в процессе последовательного возведения в квадрат y становится равным 1 до того, как оно станет равным $n - 1$, код фиксирует нетривиальный корень и объявляет число составным. Именно это обеспечивает высокую точность метода.

Итоги работы

Благодаря данной работе я на практике изучил различие между детерминированными и вероятностными алгоритмами. Несмотря на наличие теоретической вероятности ошибки (не более $1/2^t$), высокая эффективность делает эти методы лучшим выбором для проверки больших чисел. Реализация теста Миллера-Рабина позволила мне закрепить навыки программирования сложных теоретико-числовых формул.